



Année scolaire : 2025-2026

UAA12 : Développement d'un site WEB dynamique

Titulaire du cours : Lucas Embrechts

5-6^e Technique de Transition – Informatique

UAA12

Développement d'un site WEB dynamique

Nom / Prénom :

Classe :

		Pondération	Pondération finale
Q1	Implémentation du projet	/100	/50
Q2	Dossier de conception	/100	/20
Q3	Défense Orale	/100	/30

1. DÉFINITION DU PROJET

Le projet consiste à la réalisation d'un site WEB de vente en ligne. Le nom du site ainsi que les articles vendus sont laissés à l'appréciation de l'élève. Ils devront néanmoins faire l'objet d'une approbation du professeur.

Le projet comprend deux parties :

1. La réalisation d'une interface WEB (langage PHP)

Cette interface permet à l'utilisateur d'interagir avec le site selon plusieurs cas d'utilisation (voir la partie « *Description des cas d'utilisation* »).

2. La conception d'une base de données relationnelles (langage SQL)

La base de données est destinée à stocker l'ensemble des informations permettant l'interaction entre l'utilisateur et l'interface. Ainsi, cette base de données doit contenir (au minimum) :

- Les informations sur les articles en vente
- Les informations sur les commandes passées
- Les informations sur les clients (comptes)

2. DESCRIPTION DES CAS D'UTILISATION

2.1. Consulter la liste des articles en vente

Lorsque le client se rend sur la page d'accueil du site, ce dernier doit proposer la liste des articles actuellement en vente. Pour chacun, le nom, le prix unitaire ainsi qu'une image d'illustration doivent apparaître.

Des filtres ainsi que des critères de tri peuvent également être proposés pour faciliter le parcours de la liste.

2.2. Consulter les informations relatives à un article en particulier

Le client peut sélectionner un article particulier pour consulter l'ensemble de ses caractéristiques détaillées : le nom, la description, le prix unitaire, la quantité disponible ainsi que l'image d'illustration. Des variantes de l'article peuvent également apparaître (ex. couleurs, tailles, etc.).

2.3. Ajouter un article dans le panier

Le client peut ajouter un article de son choix dans un panier en précisant la quantité désirée.

La notion de « panier » est décrite au point suivant.

2.4. Consulter le contenu du panier

Un « panier » est une fonctionnalité qui permet au client de rassembler les produits qu'il souhaite acheter. Il s'agit d'un élément essentiel de toute boutique en ligne car il permet au client de retrouver facilement ses articles même après avoir fermé le navigateur. De plus, il n'est pas nécessaire de disposer d'un compte client pour profiter de cette fonctionnalité.

Le panier du site de vente en ligne doit proposer les éléments suivants :

- Une liste des articles que le client a ajoutés au panier ;
- La possibilité de modifier la quantité de chaque article ;
- Le prix total des articles ;
- Un bouton permettant au client de passer commande.

Des fonctionnalités supplémentaires peuvent être proposées pour améliorer l'expérience d'achat, telles que la possibilité de créer des listes de souhaits.

2.5. Passer une commande

Lorsque le client a validé son panier, ce dernier doit réaliser le processus de commande pour finaliser son achat. Ce processus est le suivant :

1. Pour passer commande, le client doit être connecté sur son compte client. S'il n'est pas connecté, le site de vente doit lui afficher la page de connexion (cette page est décrite au point suivant).
2. Un fois connecté, le client doit renseigner son adresse de livraison et de facturation
3. Enfin, le site doit proposer une page récapitulative : le contenu de la commande, l'adresse de livraison et de facturation ainsi que le total à payer. Cette page doit également comporter un bouton d'achat finalisant la commande.

2.6. Se connecter au compte client

Le site doit proposer un page permettant à l'utilisateur de se connecter à son compte client à l'aide d'un nom d'utilisateur et d'un mot de passe. Cette page doit également comporter un bouton qui redirige vers une autre page permettant de créer un compte.

Dès qu'un client est connecté, le contenu du panier est automatiquement lié à son compte. Ainsi, si le client se connecte sur un autre appareil, il doit pouvoir retrouver l'ensemble des articles précédemment ajoutés au panier.

2.7. Créer un compte client

Le site doit proposer une page permettant à un client de créer un compte en renseignant les informations suivantes : nom d'utilisateur, mot de passe, nom, prénom et date de naissance.

2.8. Bénéficier de promotions (bonus)

Le site peut proposer des promotions sur certains produits. Les règles de promotions applications sont laissées à l'appréciation de l'élève mais doivent être logiques et cohérentes.

3. DOSSIER DE CONCEPTION ET D'ANALYSE DU PROJET

Ce dossier reprendra :

1. La définition du projet
2. La description des données
3. Perspectives et rétrospective

3.1. Définition du projet

Cette section reprend la définition complète du projet. En d'autres termes :

- quels sont les articles en vente ?
- y a-t-il une charte graphique bien définie ?
- y a-t-il des catégories regroupant ces articles ?
- y a-t-il des règles de promotions sur ces articles qui sont applicables ?
- ...

3.2. La description des données

La base de données doit être documentée :

- i. un schéma e-a
- ii. un schéma relationnel
- iii. la description des colonnes de toute des tables (nom de la colonne, type, obligatoire/facultatif, clé primaire ou non, clé étrangère ou non en précisant vers quelle colonne)

3.3. Rétrospective

Les forces, les faiblesses de votre projet. Si certaines fonctionnalités n'ont pas pu être implémentées, expliquer les raisons.

3.4. Perspectives

Les améliorations du programme. Que peut-on améliorer ? Que peut-on imaginer en plus ? etc.

4. CODE SOURCE

- La plateforme doit être développée avec la technologie PHP / Javascript / CSS / HTML.
- La plateforme doit être dynamique c'est-à-dire alimentée par une base de données MySQL.
- La plateforme doit reposer principalement sur des traitements côté serveur, ce qui implique une prédominance du code en PHP par rapport à JavaScript.
- L'entièreté du code doit être développé par les membres de l'équipe. Chaque fichier doit présenter un commentaire en en-tête avec le nom de la personne ayant codé ce fichier (si cette partie a été codée en duo, les noms des deux, auteurs doivent être indiqués).
- En tout temps, l'interface présentera un pied de page (*footer*) fixe reprenant les noms des membres de l'équipe, le nom du projet et l'année académique. Le pied de page doit apparaître sur toutes les pages de la plateforme, au bas de l'écran, sans gêner l'affichage du contenu principal.
- Les parties plus complexes du code doivent être commentées de manière à permettre à un utilisateur lambda de comprendre et de reprendre facilement le code. C'est-à-dire que pour chaque partie de code répondant à une tâche spécifique, le développeur doit décrire en commentaire ce que le code réalise. Tout commentaire inutile sera sanctionné dans la cotation. Le commentaire doit être précis, pertinent et apporter un éclairage utile sur la logique de la tâche.
- La plateforme et la base de données doivent pouvoir fonctionner en local sur l'outil XAMPP.
- L'ensemble du code source doit être hébergé dans un dépôt GIT sur la plateforme GitHub ; en tout temps, ce dépôt doit être accessible au professeur.
- Une architecture bien spécifique doit être respectée :

```
2606_56TT_Nom(s)_WebShopCodeSource/  
├── database/  
│   ├── script_ddl.sql  
│   └── script_dml.sql  
└── website/  
    ├── css  
    ├── res  
    ├── js  
    ├── index.php  
    ├── pageInscription.php  
    └── ...
```

5. PLANIFICATION DU PROJET

Le projet débute la première semaine de cours après les congés d'hiver et se déroulera jusqu'à la fin de l'année scolaire.

La planification complète est disponible dans un document dédié sur Classroom.

6. SYSTÈME D'ÉVALUATION

6.1. Evaluation du travail journalier

Tout au long de l'année, plusieurs remises intermédiaires seront organisées et constitueront l'évaluation du TJ :

1. Dossier de conception : définition du projet
2. Dossier de conception : description des données
3. Dossier de conception : dossier complet (version intermédiaire)
4. Présentation du projet en groupe

Attention, le TJ ne se limite pas aux remises : l'implémentation du projet pendant les séances seront aussi évaluées.

6.2. Evaluation sommative

La remise finale du projet constitue l'évaluation de l'UAA12.

Le projet est évalué sur base :

- d'une défense orale (30%)
- du dossier de conception (20%)
- du code source (50%)

Les différentes grilles d'évaluation sont disponibles à la fin de ce document.

Évaluation du dossier de conception

Le dossier de conception doit être complet. La mise en page, l'orthographe et la qualité de la rédaction seront prises en compte dans l'évaluation.

Évaluation du programme

Le programme sera évalué selon les points suivants :

- le programme s'exécute correctement au niveau des fonctionnalités attendues,
- le code source est de qualité.

Des bonus seront attribués pour les fonctionnalités supplémentaires non mentionnées dans le dossier.

Toutefois, ces bonus ne seront attribués que si la note du travail de base atteint les 80%.

Évaluation orale

L'évaluation orale consiste à présenter le projet oralement devant les superviseurs. La présentation se déroulera comme suit :

1. Présentation du logiciel

2. Présentation du code source

3. Questions / Réponses sur le logiciel, le code source et le cahier des charges

La présentation du logiciel ainsi que la présentation du code source ne devra pas dépasser 10 minutes.

Dans le cas où le projet est réalisé par deux élèves, ces derniers doivent impérativement démontrer leur participation au projet. À cette fin, le code source devra être documenté de telle manière que l'on puisse identifier clairement l'auteur de chaque ligne de code.

Lors de l'épreuve orale, les questions sur le code source ne porteront que sur le code écrit par l'élève. Par contre, les questions sur le cahier des charges porteront sur l'entièreté de ce dernier.

Attention : le niveau d'exigence sera supérieur pour les projets réalisés à deux, tant au niveau de la rédaction du cahier des charges qu'au niveau de la réalisation du programme.

Les différentes grilles d'évaluation sont disponibles à la fin de ce document.

7. REMISE FINALE ET NOMENCLATURES

Lors de la remise finale vous remettrez un archive compressée **2606_56TT_Nom(s)_WebShop.zip** comprenant :

1. Le dossier de conception et d'analyse du projet en .pdf nommé **2606_56TT_Nom(s)_WebShopDossierConception.pdf**
2. Un dossier contenant votre projet complet. Ce dossier présentera la nomenclature **2606_56TT_Nom(s)_WebShopCodeSource** et contiendra :
 - I. un dossier nommé **database** contenant les scripts SQL (DDL et DML) de la base de données
 - II. un dossier nommé **website** contenant :
 - i. les pages PHP (dont l'index qui contiendra la page d'accueil)
 - ii. l'ensemble des sous dossiers suivants :
 - **css** : dossier contenant les différents fichiers de style
 - **res** : dossier contenant les fichiers ressources (images, .txt, polices, ...)
 - **js** : dossier contenant les différents fichiers javascript

8. MODALITÉS DE RÉCUPÉRATION

8.1. Conditions d'accès

La récupération est autorisée si :

- L'élève a participé au projet et a remis une première version exploitable (même incomplète).
- Les manquements relèvent d'améliorations possibles (fonctionnalités, dossier, préparation orale) et non d'une absence de travail.

8.2. Éléments à représenter

Selon les lacunes constatées, la récupération peut porter sur un ou plusieurs éléments :

- Le code / fonctionnalités attendues.
- Le dossier de conception (mise à jour et corrections).
- La défense orale individuelle.

8.3. Exigences minimales

- La version représentée doit corriger explicitement les points signalés lors de la première évaluation.
- Toute amélioration doit rester cohérente avec le cahier des charges et la méthodologie de travail demandée.
- Les livrables sont remis avec la même rigueur de présentation (structure, clarté, orthographe, etc.).

9. INDICATEURS DE QUALITÉ

9.1. La capacité fonctionnelle

« Le programme fait-il ce qu'il est censé faire ? »

La capacité fonctionnelle est la capacité qu'ont les fonctionnalités d'un logiciel à répondre aux exigences et besoins explicites ou implicites des usagers. En font partie la précision, l'interopérabilité, la conformité aux normes et la sécurité.

Exemple : Dans un jeu de Snake , si le serpent mange une pomme, son corps doit s'allonger d'un bloc et le score doit augmenter de 1 point. Si le score ne bouge pas, la capacité fonctionnelle est défailante.

9.2. La facilité d'utilisation

« Comment utiliser le programme sans devoir lire un manuel de 50 pages ? »

Elle porte sur l'effort nécessaire pour apprendre à manipuler le logiciel. En font partie la facilité de compréhension, d'apprentissage et d'exploitation et la robustesse. Un autre exemple, une utilisation incorrecte n'entraîne pas de dysfonctionnement.

Exemple : Au lieu d'écrire « Appuyez sur la touche 34 pour sauter », utilisez les flèches du clavier ou les touches ZSQD qui sont naturelles pour un joueur. L'ajout d'un bouton « Rejouer » bien visible après une partie perdue améliore aussi cet indicateur.

9.3. La fiabilité

« Le programme plante-t-il ou s'arrête-t-il brusquement ? »

C'est-à-dire la capacité d'un logiciel de rendre des résultats corrects quelles que soient les conditions d'exploitation. En font partie la tolérance aux pannes - la capacité d'un logiciel de fonctionner même en étant handicapé par la panne d'un composant (logiciel ou matériel).

Exemple : Si un élève entre son nom dans le menu et qu'il tape des symboles bizarres (comme @\$%), le jeu ne doit pas s'éteindre ou afficher une page blanche. Il doit rester stable quelles que soient les actions imprévues de l'utilisateur.

9.4. La performance

« Le programme est-t-il une usine à gaz ? »

C'est-à-dire le rapport entre la quantité de ressources utilisées (moyens matériels, temps, personnel), et la quantité de résultats délivrés. En font partie le temps de réponse, le débit et l'extensibilité - capacité à maintenir la performance même en cas d'utilisation intensive.

Exemple : Si un jeu met 10 secondes à charger une image ou si le personnage se déplace avec des saccades (le « lag »), la performance est mauvaise. Un bon code JavaScript doit permettre au jeu de tourner de façon fluide, même sur un vieil ordinateur de l'école.

9.5. La maintenabilité

« Sera-t-il facile de modifier ou d'améliorer le code plus tard ? »

Elle mesure l'effort nécessaire à corriger ou transformer le logiciel. En font partie l'extensibilité, c'est-à-dire le peu d'effort nécessaire pour y ajouter de nouvelles fonctions.

Exemple : Si vous avez utilisé une variable `vitesse = 5` au début de votre code, vous pouvez changer la difficulté du jeu en modifiant un seul chiffre. Si vous avez écrit « 5 » manuellement à 100 endroits différents dans votre code, votre projet n'est pas maintenable.

9.6. La portabilité

« Le programme peut-il fonctionner partout ? »

C'est-à-dire l'aptitude d'un logiciel de fonctionner dans un environnement matériel ou logiciel différent de son environnement initial. En font partie la facilité d'installation et de configuration dans le nouvel environnement.

Exemple : un jeu doit fonctionner de la même manière si vous y jouer sur une console de Nintendo, Sony ou Microsoft.

GRILLE D'EVALUATION
IMPLEMENTATION DU PROJET

Grille d'évaluation					
Critères	Indicateurs	Points	Acquisition		
			A	N-A	E-A
Respect des normes, règles et consignes	A respecté les délais (MALUS) A remis le projet selon une nomenclature correcte (MALUS)				
Maitrise technique et processus	Maitrise des techniques de programmation Le projet ne présente aucun bug L'élève a amené des éléments nouveaux (recherches) Le code est optimisé et permet sa maintenabilité Le code est de qualité				
Produit fini	Le projet représente un ensemble fonctionnel. L'élève a implémenté tous les cas d'utilisation Le projet est original, les règles graphiques sont respectées et cohérentes. Le projet est ergonomique et facile à prendre en main.				
Communication	Le projet ne présente aucune faute d'orthographe.				

Pondération	/100
-------------	------

GRILLE D'EVALUATION
DOSSIER DE CONCEPTION

Grille d'évaluation				
Critères	Indicateurs	Acquisition		
		A	N-A	E-A
Respect des normes, règles et consignes	A respecté les délais (MALUS) A remis le fichier selon une nomenclature correcte (MALUS)			
Produit fini	Le dossier est complet. - Définition du projet - Description des données - Rétrospective - Perspectives Le dossier explique les choix de conception de manière complète.			
Communication	Le dossier de conception ne comprend pas de faute d'orthographe. La mise en page, la structure des textes et leur contenu permettent une lecture facile et une compréhension totale du dossier.			
Pondération		/100		

GRILLE D'EVALUATION
DEFENSE ORALE INDIVIDUELLE

Grille d'évaluation					
Critères	Indicateurs	Points	Acquisition		
			A	N-A	E-A
Technique et processus	L'élève prouve le développement d'une part équitable du projet. L'élève peut vulgariser avec aisance le fonctionnement général de son code. L'élève maîtrise et peut restituer une explication claire d'un algorithme particulier. L'élève maîtrise les détails techniques propre aux langages de programmation utilisés. L'élève est capable de proposer une correction de son code et/ou une ou plusieurs fonctionnalité(s) supplémentaire(s).				

Pondération	/100
-------------	------

FEUILLE DE DÉFINITION DU PROJET

Nom des élèves :

.....

.....

Choix du projet (cochez le projet choisi):

Date :

Signature des élèves

Signature du professeur